



星航数伴

面向民航旅客的文旅推荐数字人智能交互 平台

产品总体设计文档

版本：V3.0

编制日期：2026 年 7 月

适用范围：软件杯 A5 赛题

目录

1. 引言	5
1.1 项目背景	5
1.2 设计目标	5
1.3 设计范围	5
1.4 术语定义	5
2. 系统架构设计	6
2.1 整体架构	6
2.2 架构层级说明	6
2.3 通信模式	6
2.4 部署架构	7
3. 模块划分与功能设计	8
3.1 前端模块	8
3.1.1 组件树结构	8
3.1.2 核心组件职责矩阵	8
3.2 后端模块	9
3.3 数字人交互模块	9
DataChannel 消息协议	9
4. 接口设计	10
4.1 RESTful API 接口规范	10
4.2 WebRTC 信令接口	10
5. 数据库设计	11

5.1 数据存储策略	11
5.2 SQLite 数据库表结构（预留）	11
5.3 JSON 知识库结构	11
5.4 前端数据模型（TypeScript 类型定义）	11
6. 关键技术选型	12
6.1 前端技术选型	12
6.2 后端技术选型	12
6.3 AI 技术选型	12
6.4 技术选型对比分析	13
7. 核心实现方案	13
7.1 数字人 WebRTC 实时通信实现	13
7.2 LLM 集成实现	13
7.3 语音输入实现	14
7.4 客流监控实现	14
7.5 管理后台实现	14
8. 数据流与通信协议	14
8.1 用户消息处理全流程	14
8.2 服务状态轮询机制	15
8.3 WebRTC 重连机制	15
9. 安全设计	15
9.1 API 安全	15
9.2 数据安全	15
9.3 WebRTC 安全	15

10. 性能设计	16
10.1 前端性能	16
10.2 后端性能	16
10.3 WebRTC 性能	16
11. 可扩展性设计	16
11.1 新增景区节点	16
11.2 新增服务阶段	17
11.3 接入新 LLM	17
11.4 扩展为多数字人	17
附录	17
A. 文件目录结构	17
B. 外部依赖清单	18
C. 版本历史	18

1. 引言

1.1 项目背景

随着民航出行普及化与文旅消费升级，旅客对“出行即服务”（Mobility as a Service, MaaS）模式的需求日益增长。当前民航与文旅系统存在信息孤岛、服务断层等问题，旅客从行前规划到返程的完整旅程缺乏统一、连贯的智能化服务体验。星航数伴平台旨在打破这一困境，通过 AI 数字人技术为旅客提供从“出门”到“回家”的全链路一体化智能导览服务。

1.2 设计目标

目标	描述
全链路覆盖	覆盖行前规划、机场导引、飞行讲解、落地景区、返程服务五大服务阶段
数字人交互	基于 WebRTC 的实时 3D 数字人音视频交互，提供沉浸式体验
智能推荐	基于大语言模型的文旅推荐与行程规划能力
实时监控	景区客流数据实时可视化，拥堵度智能预警
高可扩展	模块化架构，支持新景区、新服务阶段的快速接入

1.3 设计范围

本文档涵盖星航数伴平台的系统架构、模块设计、接口规范、数据模型、关键技术选型及核心实现方案。

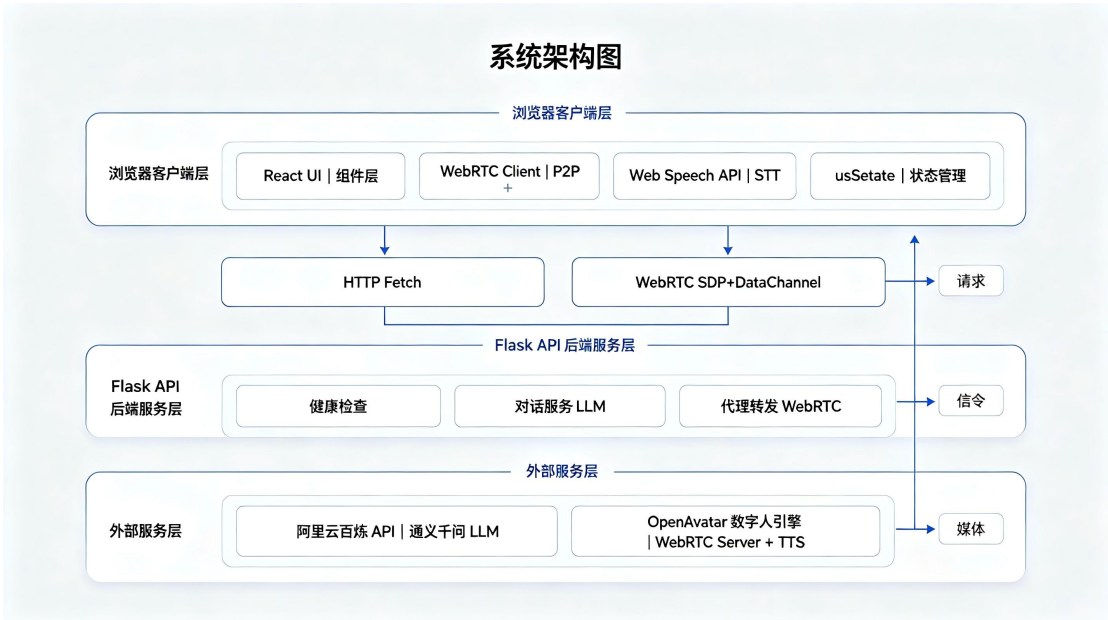
1.4 术语定义

术语	全称	说明
OpenAvatar	OpenAvatarChat	开源数字人实时渲染与驱动引擎
WebRTC	Web Real-Time Communication	浏览器实时通信协议，用于数字人音视频流传输
DataChannel	RTCDataChannel	WebRTC 数据通道，用于文本消息与指令传输
LLM	Large Language Model	大语言模型（本系统使用通义千问 qwen-plus）
SDP	Session Description Protocol	WebRTC 会话描述协议，用于协商媒体参数
STT	Speech to Text	语音转文本（使用浏览器 Web Speech API）

2. 系统架构设计

2.1 整体架构

本系统采用 B/S 三层架构+WebRTC 点对点通信的混合架构。



2.2 架构层级说明

浏览器客户端层：React UI 组件层负责界面渲染、用户交互与状态管理；WebRTC Client 建立与 OpenAvatar 引擎的点对点连接；Web Speech API 提供浏览器原生语音识别；状态管理基于 React useState Hooks。

后端服务层（Flask API）：健康检查模块提供服务状态查询；对话服务模块集成 LLM 调用与本地降级规则匹配；代理转发模块代理 WebRTC SDP Offer/Answer 交换。

AI 与数字人服务层：阿里云百炼 OpenAI 兼容接口提供通义千问（qwen-plus）大语言模型；服务器本地运行 OpenAvatarChat，组合 CosyVoice 语音合成、LiteAvatar 55 号数字人驱动与 WebRTC 媒体传输。

2.3 通信模式

通道	协议	用途	说明
HTTP REST	HTTP/1.1	状态查询、降级对话	前端与后端之间的标准 REST 通信
WebRTC P2P	WebRTC(UDP/TCP)	数字人视频流、实时对话	浏览器与 OpenAvatar 引擎间的高带宽实时通信

当前版本采用“HTTP 生成答案+WebRTC 驱动口播”的组合链路：前端将用户问题提交给 Flask 后端，后端检索本地灵山胜境知识库并调用通义千问生成答案；前端收到答案后，通过已建立的 WebRTC DataChannel 发送 SendAvatarText，触发 55 号数字人口播。LLM 不可用时，后端使用本地知识库与规则回复兜底。

2.4 部署架构

系统已部署至阿里云 ECS（Ubuntu 22.04，Tesla P100 16 GB），公网入口为 <http://8.130.94.124:3000/>。前端、Flask 后端、OpenAvatarChat 与 Coturn 均运行在同一云服务器，通义千问通过阿里云百炼 API 调用，灵山胜境资料以本地 JSON 知识库形式部署。

2.5 当前云端运行架构（2026 年 7 月）

云端版本已实现统一公网访问、知识库增强问答、55 号数字人实时画面与语音口播。浏览器只访问前端入口，业务回答由 Flask 后端统一编排，OpenAvatarChat 负责数字人媒体生成。

组件	当前部署	作用
Web 前端	React/Vite, TCP 3000	页面交互、状态展示、聊天与 WebRTC 客户端
业务后端	Flask, TCP 5001	健康检查、知识库检索、千问调用、信令代理
数字人引擎	OpenAvatarChat, TCP 8282	55 号数字人、CosyVoice、LiteAvatar、WebRTC
媒体中继	Coturn, TCP/UDP 3478	跨网络 ICE 协商与音视频中继
知识与模型	本地知识库+阿里云百炼	灵山胜境资料约 110 条事实；qwen-plus 生成回答

公网访问地址：<http://8.130.94.124:3000/>



扫码访问“星航数伴”云端体验平台

当前稳定性配置：数字人服务并发上限为 1，LiteAvatar 采用 10 FPS 快速模式。同一时刻仅支持一个完整数字人音视频会话；其他访客仍可打开页面，但需等待会话释放。

3. 模块划分与功能设计

3.1 前端模块

3.1.1 组件树结构

```
<App>[主应用-状态中心]
├─<header>[顶部导航栏]
│   └─实时时钟显示[每秒刷新]
│   └─平台标题+系统内核版本[品牌标识]
│   └─“控制台”按钮[触发 AdminOverlay]
├─<LeftPanel>[左侧面板]
│   └─HeroCard[景区名牌卡片]
│   └─InfoCard[项目简介卡]
│   └─FlowCard[实时客流监控卡]
├─<CenterPanel>[中央展示舱-数字人]
│   └─全息展示舱框架
│   └─OpenAvatar 视频元素[WebRTC 媒体流渲染]
│   └─唤醒芯片状态指示器
├─<RightPanel>[右侧面板-服务列表]
│   └─5 个 ServiceCard(行前/机场/飞行/景区/返程)
├─<ChatDock>[底部聊天区]
│   └─ChatLog(消息日志列表)
│   └─输入区域(文本+语音+发送)
└─<AdminOverlay>[管理后台弹窗]
```

3.1.2 核心组件职责矩阵

组件	职责	输入 Props	输出/副作用
App	全局状态管理、WebRTC 生命周期	无	所有子组件渲染、轮询定时器
LeftPanel	景区信息展示、客流可视化	overview, spots	静态渲染
CenterPanel	数字人视频流播放	openAvatarOnline, openAvatarStream	视频元素播放控制
RightPanel	服务阶段导航列表	stages, activeStage	阶段切换回调
ChatDock	消息展示、用户	消息、回调函数	发送消息、语音请

输入		求	
AdminOverlay	配置数据编辑	overview, spots	数据更新

3.2 后端模块

```
backend/
├─ app.py[主应用]
│  ├─ 环境配置加载(load_dotenv)
│  ├─ 路由管理器(create_app)
│  │  ├─ GET/api/health[健康检查]
│  │  ├─ GET/api/service/status[服务状态]
│  │  ├─ GET/api/openavatar/status[OA 状态检测]
│  │  ├─ GET/api/openavatar/config[OA 配置分发]
│  │  ├─ POST/api/openavatar/webrtc/offer[WebRTC SDP 代理]
│  │  ├─ POST/api/openavatar/chat[OA 同步对话]
│  │  └─ POST/api/digital-human/chat[数字人对话核心]
│  ├─ LLM 调用层(request_chat_model)
│  ├─ 降级策略层(fallback_reply)
│  └─ OpenAvatar 代理层(proxy_openavatar_request)
└─ build_scenic_kb.py[知识库构建工具]
```

后端处理流程：请求到达→Flask 路由匹配→CORS 中间件处理→业务逻辑（优先 LLM 调用，失败时降级为本地规则匹配）。

3.3 数字人交互模块

WebRTC 连接生命周期：创建 PeerConnection→添加本地模拟流→创建 DataChannel→生成 SDP Offer→POST 到后端代理→接收 SDP Answer→设置远端描述→接收远端 MediaStream→连接就绪。

重连机制：连接失败→等待 10 秒→重置连接→重新执行连接流程。

DataChannel 消息协议

消息名称	方向	说明
SendHumanText	客户端→引擎	发送用户文本，请求 AI 回复
SendAvatarText	客户端→引擎	发送 AI 回复文本，触发数字人口播
EchoHumanText	引擎→客户端	回显用户文本（确认接收）
EchoAvatarText	引擎→客户端	流式返回 AI 回复文本
ChatSignal	引擎→客户端	播放控制信号（stream_begin 等）

4. 接口设计

4.1 RESTful API 接口规范

Base URL: `http://127.0.0.1:5001/api`; 数据格式: `application/json`; 字符编码: UTF-8; 状态码遵循 HTTP 标准。

GET/api/health—健康检查

检测后端 API 服务是否在线。响应: `{"online":true,"service":"backend"}`

GET/api/openavatar/status—数字人状态

检测 OpenAvatar 引擎是否在线。返回 online 状态、baseUrl、chatUrl 等。

GET/api/openavatar/config—数字人配置

获取 OpenAvatar 完整连接配置信息，供前端初始化使用。

POST/api/openavatar/webrtc/offer—WebRTC 信令代理

代理转发浏览器发出的 WebRTC SDP Offer，接收引擎返回的 SDP Answer。

POST/api/digital-human/chat—数字人对话（核心）

支持按服务阶段上下文提供智能回复。请求体: message（必填）、stage、inputMode、context。

POST/api/openavatar/chat—OpenAvatar 同步对话

与 OpenAvatar 引擎同步对话，返回 AI 回复并触发数字人口播。

4.2 WebRTC 信令接口

SDP 交换流程: 浏览器创建 Offer→setLocalDescription→POST 到后端代理→后端转发到 OpenAvatar 引擎→接收 Answer→返回浏览器→setRemoteDescription→WebRTC 连接建立。

DataChannel 消息格式统一为 JSON:

```
{
  "header": {
    "name": "SendHumanText",
    "request_id": "uuid"
  },
  "payload": {
    "request_id": "uuid",
    "stream_key": "uuid",
    "mode": "full_text",
    "text": "用户消息内容",
    "end_of_speech": true
  }
}
```

5. 数据库设计

5.1 数据存储策略

存储类型	技术	用途	说明
关系型数据库	SQLite	数字人元数据管理	memory/fay.db
JSON 文件存储	JSON	景区知识库	memory/scenic_kb/*.json
运行时状态	React useState	UI 状态、对话历史	仅内存，刷新后重置

5.2 SQLite 数据库表结构（预留）

```
CREATE TABLE IF NOT EXISTS digital_humans(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name VARCHAR(120) NOT NULL,  
  model_type VARCHAR(40),  
  model_url VARCHAR(500),  
  avatar_url VARCHAR(500),  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

字段	类型	说明
id	INTEGER	主键，自增
name	VARCHAR(120)	数字人角色名称
model_type	VARCHAR(40)	模型类型标识
model_url	VARCHAR(500)	3D 模型资源 URL
avatar_url	VARCHAR(500)	数字人形象图片 URL
created_at	DATETIME	创建时间戳

说明：当前版本中数据库为预留设计，数字人配置通过 OpenAvatar 引擎的 YAML 配置文件管理。

5.3 JSON 知识库结构

景区资料知识库（imported_chunks.json）包含 generatedFrom（来源目录）、documents（文档数组），每个文档含 source、sizeBytes 和 chunks（id+content 对象数组）。结构化知识库（lingshan_knowledge.json）包含景区的结构化事实数据。

5.4 前端数据模型（TypeScript 类型定义）

```
// 服务阶段  
type ServiceStage={
```

```
id:string;title:string;code:string;
description:string;status:string;
statusType:"good"|"busy";
};
//景区节点
type ScenicSpot={
id:string;name:string;code:string;
description:string;currentVisitors:number;
maxCapacity:number;
status:"推荐"|"适中"|"繁忙";
};
//拥堵度
type ComfortLevel="畅通"|"适中"|"拥挤";
```

6. 关键技术选型

6.1 前端技术选型

技术	版本	选型理由
React	18.2	生态成熟、函数组件+Hooks 模式高效、社区资源丰富
TypeScript	5.2	静态类型检查提升代码可靠性，接口定义清晰
Vite	5.1	极速冷启动、HMR 热更新、原生 ESM 支持
纯 CSS	-	无第三方 UI 框架，自定义暗色科技风主题，减少依赖

6.2 后端技术选型

技术	版本	选型理由
Flask	3.0	轻量级、灵活路由、适合 API 服务场景
Flask-CORS	4.0	自动处理跨域请求，简化前后端分离部署
Requests	2.31	Python 生态标准 HTTP 客户端库

6.3 AI 技术选型

技术	选型理由
通义千问 qwen-plus	中文理解能力强、OpenAI 兼容 API 格式、响应速度快、成本可控
OpenAvatarChat	开源数字人引擎、支持 WebRTC 实时通信、可本地部署
Web Speech API	浏览器原生、无需额外服务端、支持中文识别

6.4 技术选型对比分析

LLM 选型对比：

方案	优势	劣势	结论
通义千问 (qwen-plus)	中文优秀、API 稳定、低成本	需联网	选用
本地部署模型	离线可用、数据安全	硬件要求高、推理慢	不选
ChatGPT API	能力强、生态好	访问受限、成本高	不选

数字人方案对比：

方案	优势	劣势	结论
OpenAvatarChat	开源、WebRTC 原生支持、可定制	需要 GPU、部署复杂	选用
商业数字人 SDK	开箱即用、效果好	成本高、定制受限	不选
纯前端动画	部署简单	效果差、无真实交互	不选

7. 核心实现方案

7.1 数字人 WebRTC 实时通信实现

连接建立流程：浏览器创建 `RTCPeerConnection` 与 `DataChannel`→生成 SDP Offer→后端代理转发至 `OpenAvatarChat`→接收 Answer 并设置远端描述→通过 `Coturn` 完成跨网络 ICE 协商→接收数字人音视频轨道。`DataChannel` 打开后，前端方可发送 `SendAvatarText` 指令。

回答与口播机制：用户输入→`POST /api/digital-human/chat`→本地知识库检索→通义千问生成回答→前端显示回答→通过 `DataChannel` 发送 `SendAvatarText`→`CosyVoice` 合成语音→`LiteAvatar` 生成 55 号数字人口型与画面→WebRTC 返回音视频。

降级策略：后端对话服务与数字人媒体链路解耦。即使 WebRTC 暂时不可用，文本回答仍可通过 HTTP 正常返回；若通义千问调用失败，则使用本地知识库与规则回复；待 `DataChannel` 恢复后再触发数字人口播。

7.2 LLM 集成实现

System Prompt：“你是民航文旅数字人向导，用自然、简洁、可执行的中文回答。”

上下文注入：每次对话请求将当前服务阶段作为上下文注入，messages 数组包含 system 角色（向导人设）和 user 角色（当前服务阶段+用户问题），temperature 设为 0.7。

本地降级规则：

关键词	回复模板
航班/机票/登机等	可以。我会结合出发地、目的地、时间和预算，给出航班筛选、值机、安检和登机提醒。
景区/路线/讲解等	可以。我会按你的兴趣和时间安排景区路线，并提醒拥挤度、讲解重点和返程衔接。
返程	收到。我会优先处理返程航班、机场交通、值机时间和行李提醒。
其他	收到。我可以继续帮你处理民航出行、机场导引、景区讲解和返程服务。

7.3 语音输入实现

基于浏览器 Web Speech API (SpeechRecognition)：检测浏览器支持→创建实例→设置 lang:zh-CN, continuous:false, interimResults:true→开始监听→onresult 事件实时更新输入框文本→finalTranscript 自动提交消息→onerror/onend 重置状态。

7.4 客流监控实现

拥堵度=当前游客数/最大容量。 $\text{ratio} < 0.45$ =畅通； $0.45 \leq \text{ratio} < 0.75$ =适中； $\text{ratio} \geq 0.75$ =拥挤。总游客数= $\sum$ 各节点 currentVisitors，总容量= $\sum$ 各节点 maxCapacity，整体拥堵度=getComfort(总游客数, 总容量)。

7.5 管理后台实现

采用 React 单向数据流：AdminOverlay 修改→onOverviewChange/onSpotsChange 回调→App 组件 setState 更新→重新渲染所有依赖组件。数据仅存在于内存中，页面刷新后恢复为 src/data.ts 中的默认值。

8. 数据流与通信协议

8.1 用户消息处理全流程

用户输入（文本或语音）→App.tsx sendText→POST /api/digital-human/chat→后端进行服务阶段识别与灵山胜境知识库检索→调用通义千问→返回答案并写入聊天区→若 WebRTC DataChannel 为 open，则发送 SendAvatarText 触发数字人口播；若媒体链路不可用，仅保留文本回答并提示等待数字人连接。

8.2 服务状态轮询机制

轮询目标	间隔	端点	用途
后端 API 服务	12 秒	/api/health	检测后端服务是否在线
OpenAvatar 引擎	5 秒	/api/openavatar/status	检测数字人引擎是否在线

8.3 WebRTC 重连机制

连接状态监控：OpenAvatar 检测在线→触发 WebRTC 连接（成功保持，失败 10 秒后重试）；OpenAvatar 检测离线→清理 WebRTC 资源→等待重新上线→自动重连。

9. 安全设计

9.1 API 安全

措施	实现方式
API 密钥保护	敏感密钥存储在 .env 文件，不纳入版本控制（.gitignore）
最小化暴露	演示环境通过安全组开放必要端口；生产环境应使用 Nginx 反向代理并限制源 IP
代理转发	OpenAvatar WebRTC 信令通过后端代理，隐藏引擎直接端口
CORS 控制	Flask-CORS 中间件配置跨域白名单

9.2 数据安全

对话历史仅存在于浏览器内存，不持久化到磁盘；API 密钥仅在服务端使用调用 LLM API，不暴露到前端代码；生产部署使用环境变量注入，不硬编码。

9.3 WebRTC 安全

WebRTC 使用 DTLS-SRTP 加密传输音视频，DataChannel 基于 SCTP over DTLS 传输控制消息；Coturn 负责跨网络中继。当前演示地址使用 HTTP 公网访问，正式生产环境应配置域名、HTTPS 与 Nginx 反向代理，并限制 5001、8282 等内部端口的公网暴露。

10. 性能设计

10.1 前端性能

优化项	实现方式
组件粒度	细粒度组件拆分，减少不必要的重渲染
条件渲染	非核心组件按需渲染（AdminOverlay 仅在打开时渲染）
定时器管理	轮询定时器在组件卸载时清理，防止内存泄漏
CSS 优化	22.7 KB 单文件样式，利用 CSS 变量和现代特性
响应式	通过媒体查询适配不同屏幕尺寸

10.2 后端性能

优化项	实现方式
无状态 API	每个请求独立处理，无需会话管理
超时控制	LLM API 超时 30 秒，OpenAvatar 代理超时 30 秒
状态缓存	轮询间隔 5 秒，避免频繁探测
最小化依赖	仅 3 个 Python 第三方包

10.3 WebRTC 性能

优化项	实现方式
视频帧率（当前 10 FPS）	OpenAvatar 配置限定为 15 FPS，降低带宽消耗
缓冲池	音频/视频缓冲池均设为 30，平衡延迟与流畅度
自动重连	连接断开 10 秒后重试，避免频繁重连
本地模拟流	使用静音音频+黑帧视频的轻量级本地流

11. 可扩展性设计

11.1 新增景区节点

在 `src/data.ts` 的 `scenicSpots` 数组中添加新的 `ScenicSpot` 对象；在 `memory/scenic_kb/` 目录添加对应的知识库 JSON 文件；通过管理后台或直接编辑添加客流数据。

11.2 新增服务阶段

在 `src/data.ts` 的 `serviceStages` 数组中添加新的 `ServiceStage` 对象；在 `backend/app.py` 的 `fallback_reply` 函数中添加对应的关键词匹配规则；在 LLM System Prompt 中补充新阶段的服务描述。

11.3 接入新 LLM

后端 LLM 调用使用 OpenAI 兼容 API 格式，更换模型只需修改 `.env` 中的 `CHAT_BASE_URL`、`CHAT_API_KEY` 和 `CHAT_MODEL` 配置项。

11.4 扩展为多数字人

数据库 `digital_humans` 表已预留多数字人管理结构。后续可通过在管理后台添加数字人配置、前端提供切换选择器、WebRTC 连接到不同 `OpenAvatar` 实例来实现。

附录

A. 文件目录结构

```
113/
├──src/#前端源码
│  ├──main.tsx#应用入口
│  ├──App.tsx#主应用组件（745 行）
│  ├──api.ts#API 通信层（93 行）
│  ├──data.ts#数据模型（135 行）
│  ├──index.css#全局样式（1283 行）
│  └──components/
│     ├──LeftPanel.tsx#左侧面板（49 行）
│     ├──CenterPanel.tsx#中央展示舱（79 行）
│     ├──RightPanel.tsx#右侧面板（38 行）
│     ├──ChatDock.tsx#聊天交互区（86 行）
│     └──AdminOverlay.tsx#管理后台（135 行）
├──backend/#后端源码
│  ├──app.py#主应用（215 行）
│  ├──build_scenic_kb.py#知识库构建（104 行）
│  ├──requirements.txt#依赖清单
│  └──schema.sql#数据库表结构
├──memory/#持久化数据
├──public/assets/#静态资源
├──index.html#HTML 入口
├──package.json#前端依赖
├──vite.config.ts#Vite 配置
├──.env/.env.example#环境变量
├──start.cmd#前端启动
└──start-backend.cmd#后端启动
```

└─start-openavatar.ps1#数字人启动

B. 外部依赖清单

类别	名称	版本	用途
运行时	react	^18.2.0	UI 框架
运行时	react-dom	^18.2.0	React DOM 渲染
运行时	flask	>=3.0.0	Web 框架
运行时	flask-cors	>=4.0.0	跨域支持
运行时	requests	>=2.31.0	HTTP 客户端
开发	typescript	^5.2.2	类型系统
开发	vite	^5.1.0	构建工具
外部	OpenAvatarChat	-	数字人渲染引擎
外部	阿里云百炼 API	qwen-plus	大语言模型

C. 版本历史

版本	日期	变更说明
V1.0	-	初始版本
V2.0	-	集成 OpenAvatar WebRTC 实时通信
V3.0	2026.07	WebRTC 流式回复、服务降级、管理后台完善